

Ensemble Learning

Dr. Huseyin Kusetogullari

Learning Outcomes

- Motivation
- Why do we need ensemble learning?
- Ensemble Methods
 - Max Voting
 - Averaging
 - Weighted Averaging
- Advanced Ensemble Methods
 - Bagging
 - Boosting
- Decision Trees
- Random Forest

Motivation

- TWO HEADS ARE BETTER THAN ONE.
- The machine learning model can be improved by combining more than one machine learning method.
- Combinations of models are generally known as model ensembles.
- The main motivations are coming from:
 - Computational Learning Theory
 - Statistics

Motivation

- The main purpose of ensemble learning is to train more than one machine learning method by using the same training data.
- Slightly different models can be obtained.

Motivation

- In essence, ensemble models in machine learning have the following two things in common:
 - They construct multiple, diverse predictive models from adapted versions of the training data.
 - They combine the predictions of these models in some way, often by simple averaging or voting.

Example

- When you want to buy a new car, will you go directly to buy the first car shop and buy one based on the advice of the dealer? It's highly not.
 - You would likely search several web pages where people have posted their reviews and compare different car models, checking for their features and prices.
 - You will also probably ask your friends and colleagues for their opinion.
- As a result, you wouldn't quickly make a decision, but will make a decision considering the opinions of other people as well.

Ensemble Methods

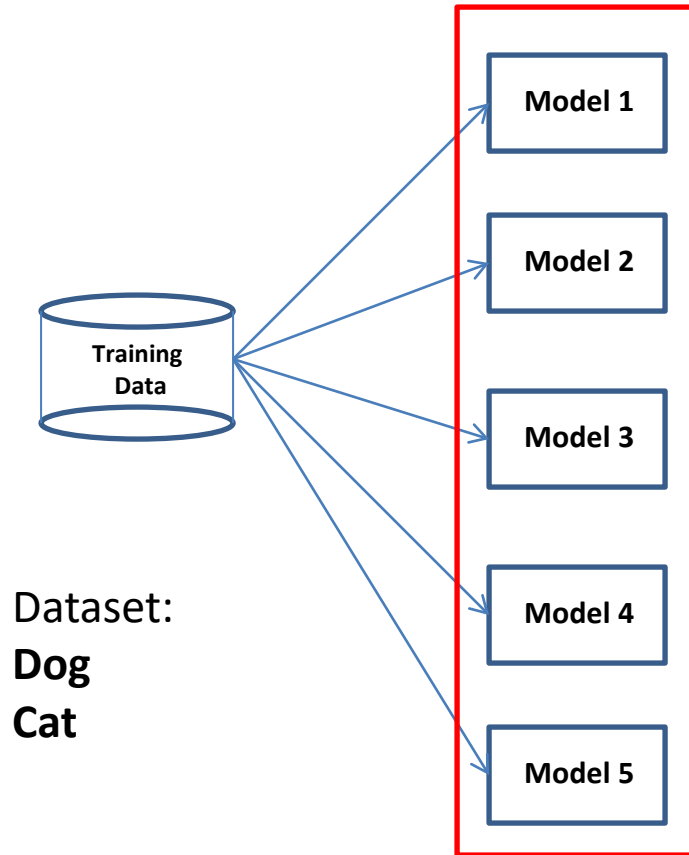
- **Simple Ensemble Techniques**
- A few simple but powerful techniques, namely:
 - Max Voting
 - Averaging
 - Weighted Averaging
- There are two other well-known ensemble methods:
 - **Bagging**
 - **Boosting**

Max Voting

- The max voting method is generally used for classification problems.
- In this technique, multiple models are used to make predictions for each data point.
- The predictions by each model are considered as a 'vote'. The predictions which we get from the majority of the models are used as the final prediction.

Example

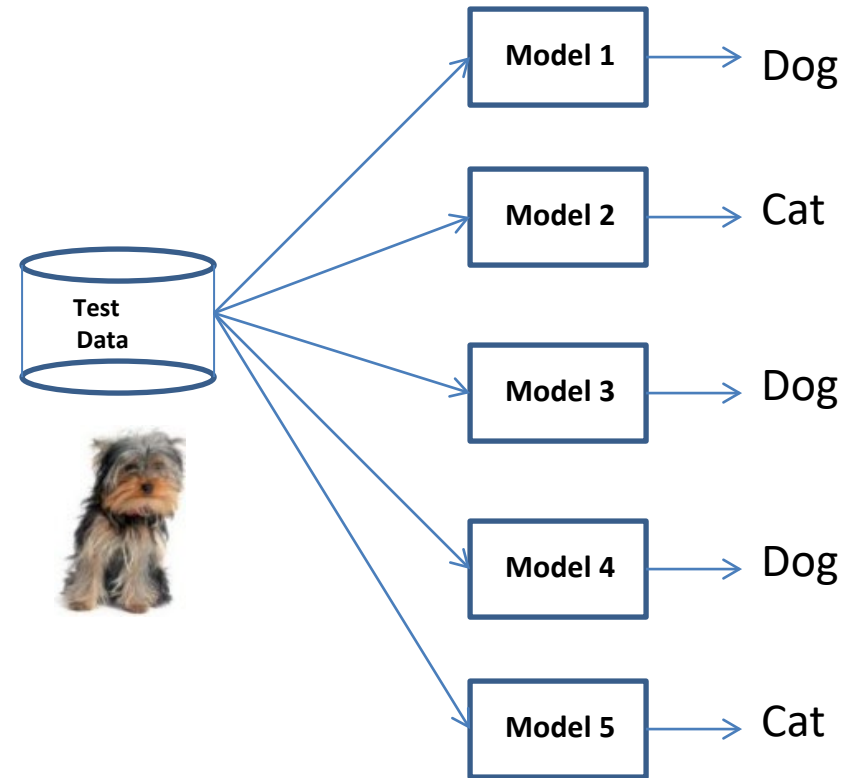
Ensemble



Dataset:
Dog
Cat

Cats

Dogs



Vote: 3 Dogs, 2 cats

Prediction Result is **Dog**

Sample Code in Python

```
model1 = tree.DecisionTreeClassifier()  
model2 = KNeighborsClassifier()  
model3= LogisticRegression()
```

```
model1.fit(x_train,y_train)  
model2.fit(x_train,y_train)  
model3.fit(x_train,y_train)
```

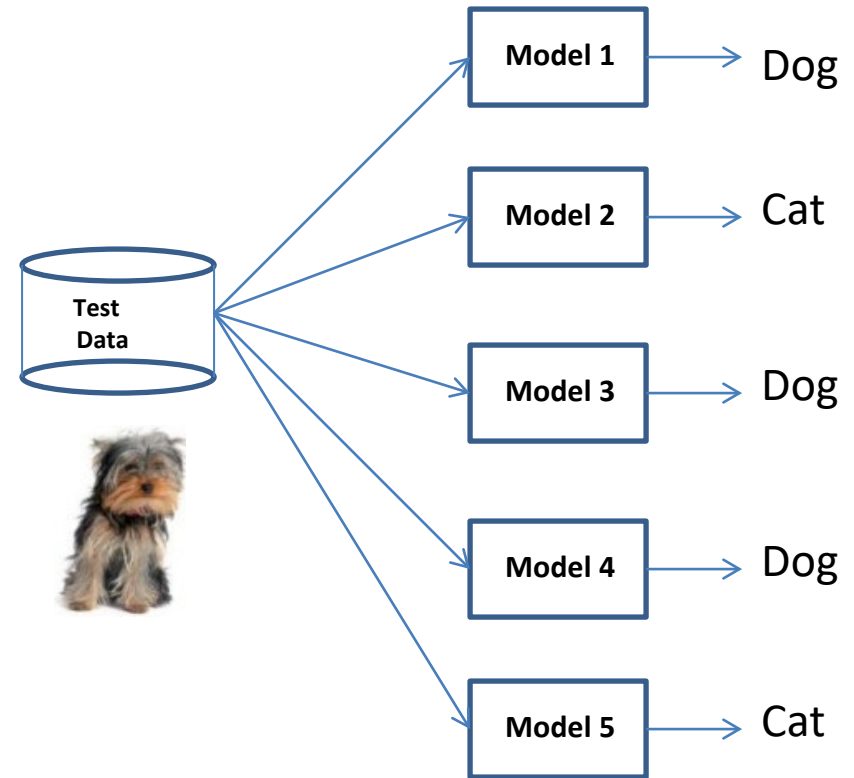
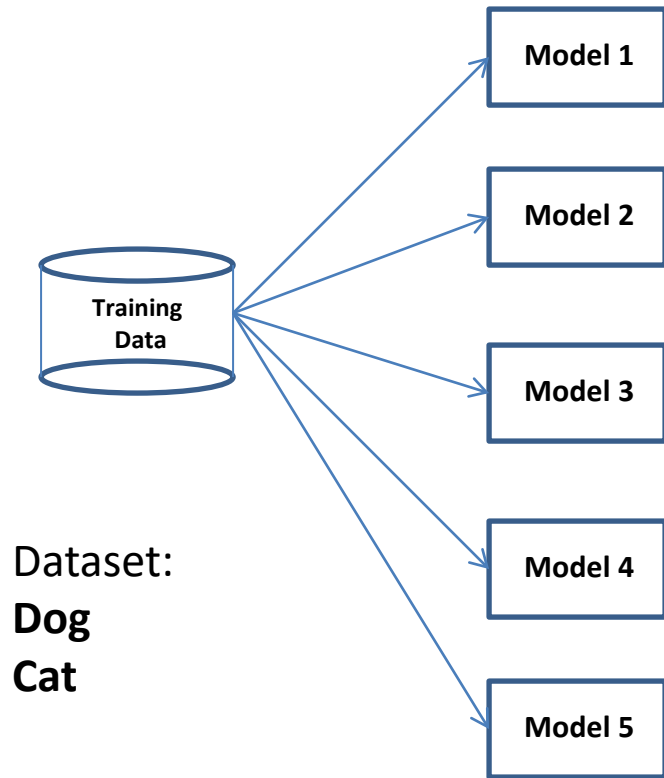
```
pred1=model1.predict(x_test)  
pred2=model2.predict(x_test)  
pred3=model3.predict(x_test) final_pred = np.array([])
```

```
for i in range(0,len(x_test)):  
    final_pred = np.append(final_pred, mode([pred1[i], pred2[i], pred3[i]]))
```

Average Voting

- Similar to the max voting technique, multiple predictions are made for each data point in averaging.
- In this method, we take an average of predictions from all the models and use it to make the final prediction.
- Averaging can be used for making predictions in regression problems or while calculating probabilities for classification problems.

Example



Vote: 3 Dogs, 2 cats

Prediction Result is Dog 60% ($3/5 = 0.6$)
Cat 40% ($2/5 = 0.4$)

Example

- Lets assume that we want to predict the tomorrow's temperature in a city. 5 different machine learning models are trained using last 5 years data and the models predict the results as follows:
- | Method 1 | Method 2 | Method 3 | Method 4 | Method 5 |
|----------|----------|----------|----------|----------|
| 25 | 21 | 18 | 27 | 28 |
- Final result is estimated as follows:
 $25+21+18+27+28/5 = \mathbf{23.8}$

Sample Code in Python

```
model1 = tree.DecisionTreeClassifier()  
model2 = KNeighborsClassifier()  
model3= LogisticRegression()
```

```
model1.fit(x_train,y_train)  
model2.fit(x_train,y_train)  
model3.fit(x_train,y_train)
```

```
pred1=model1.predict(x_test)  
pred2=model2.predict(x_test)  
pred3=model3.predict(x_test) final_pred = np.array([])
```

```
finalpred=(pred1+pred2+pred3)/3
```

Weighted Average

- This is an extension of the averaging method.
- All models are assigned with different weights defining the importance of each model for prediction.
- For instance, , based on your experience, you can think that the first model can provide better than the second model. Therefore, the first model will have higher weight value than the second one.

Example

- Lets assume that we want to predict the tomorrow's temperature in a city. 5 different machine learning models are trained using last 5 years data and the models predict the results as follows:

•	Weight1	Weight2	Weight3	Weight4	Weight5
	0.15	0.25	0.15	0.20	0.25
•	Method 1	Method 2	Method 3	Method 4	Method 5
	25	21	18	27	28

- Final result is estimated as follows:

$$(25*0.15)+(21*0.25)+(18*0.15)+(27*0.20)+(28*0.25) = \mathbf{24.1}$$

Sample Code in Python

```
model1 = tree.DecisionTreeClassifier()  
model2 = KNeighborsClassifier()  
model3= LogisticRegression()
```

```
model1.fit(x_train,y_train)  
model2.fit(x_train,y_train)  
model3.fit(x_train,y_train)
```

```
pred1=model1.predict(x_test)  
pred2=model2.predict(x_test)  
pred3=model3.predict(x_test) final_pred = np.array([])
```

```
finalpred=(0.3*pred1+0.3*pred2+0.4*pred3)
```

Algorithm

- Train an ensemble of models

- **Input:** Training Data D , ensemble size T , learning Algorithm A
- **Output:** ensemble of models whose predictions are to be combined by voting or averaging

For $t=1$ to T do

 run A_t on D to produce a model M_t ;

End

Return $\{M_t \mid 1 \leq t \leq T\}$

- Test

- **Input:** Test the Data K

For $t=1$ to T do

 run M_t on K to predict a result;

End

Final_result $\rightarrow$ Apply voting approach on *predicted results*

Return $\{\mathbf{final_result}\}$

QUESTION...

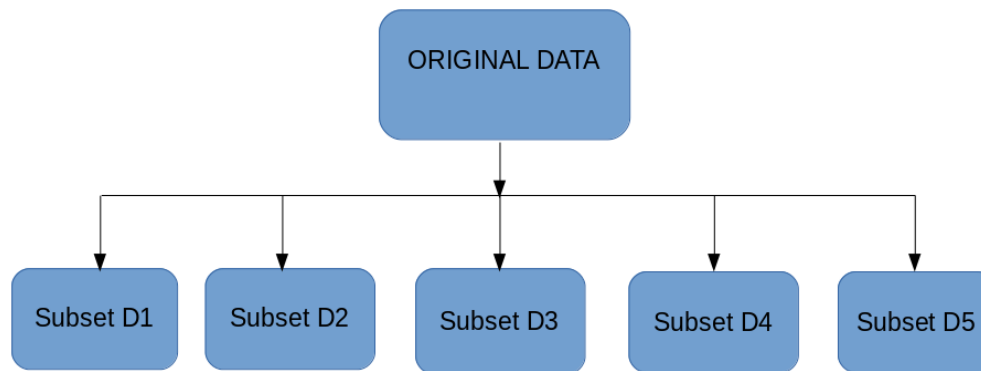
- If all models are created using the same set of data and combine it, will it be useful?
- There is a high chance that these models will give the same result since they are getting the same input. So how can we solve this problem?
- One of the techniques is bootstrapping.

Bagging and Random Forests

- Bagging:
 - Highly effective ensemble method that creates diverse models on different random samples of the original data set.
 - The samples are taken uniformly with replacement and are known as *bootstrap* samples.
 - Each subset of data of the original data is used to create or train different model in the ensemble.
 - Thus, different models will be obtained in the ensemble which we want to do it.

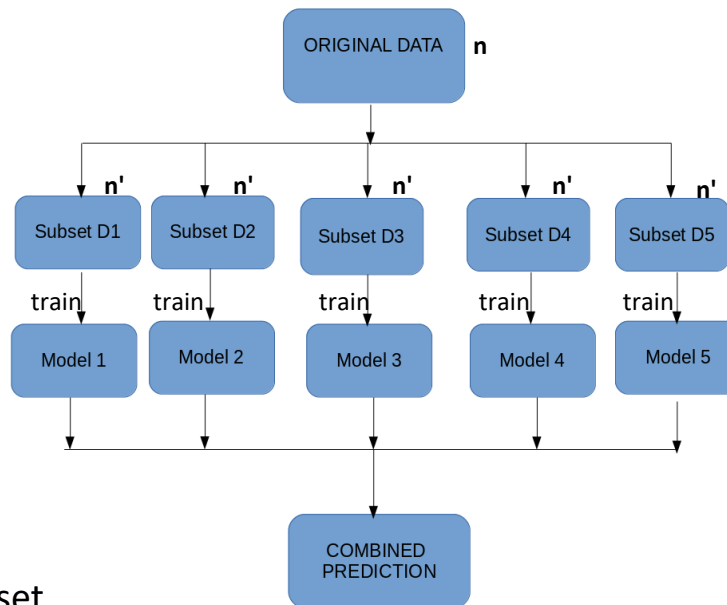
Bagging

- The idea behind bagging is combining the results of multiple models (for instance, all decision trees) to get a generalized result.
- Bootstrapping is a sampling technique in which we create subsets of observations from the original dataset, **with replacement**.



Bagging

- Multiple subsets are created from the original dataset, selecting observations with replacement.
- A base model (weak model) is created on each of these subsets.
- The models run in parallel and are independent of each other.
- The final predictions are determined by combining the predictions from all the models.



$n' < n$ (60% of training data)

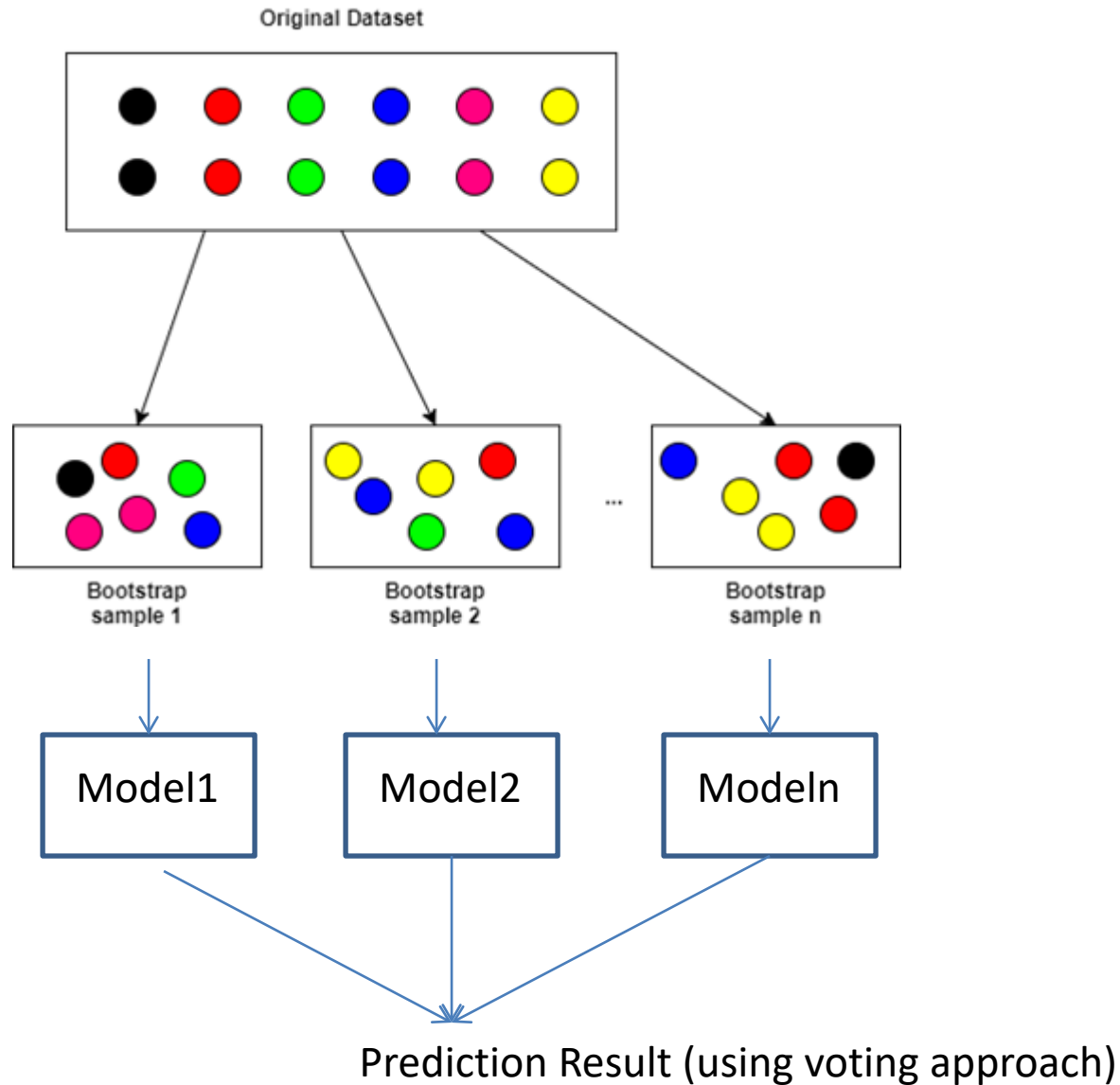
n : Size of dataset

n' : size of each subset dataset

Advantages

- Bootstrap Aggregation or bagging is a powerful technique that
 - reduces model variances (overfitting),
 - improves the outcome of learning on limited sample (i.e. small number of observations) or unstable datasets.

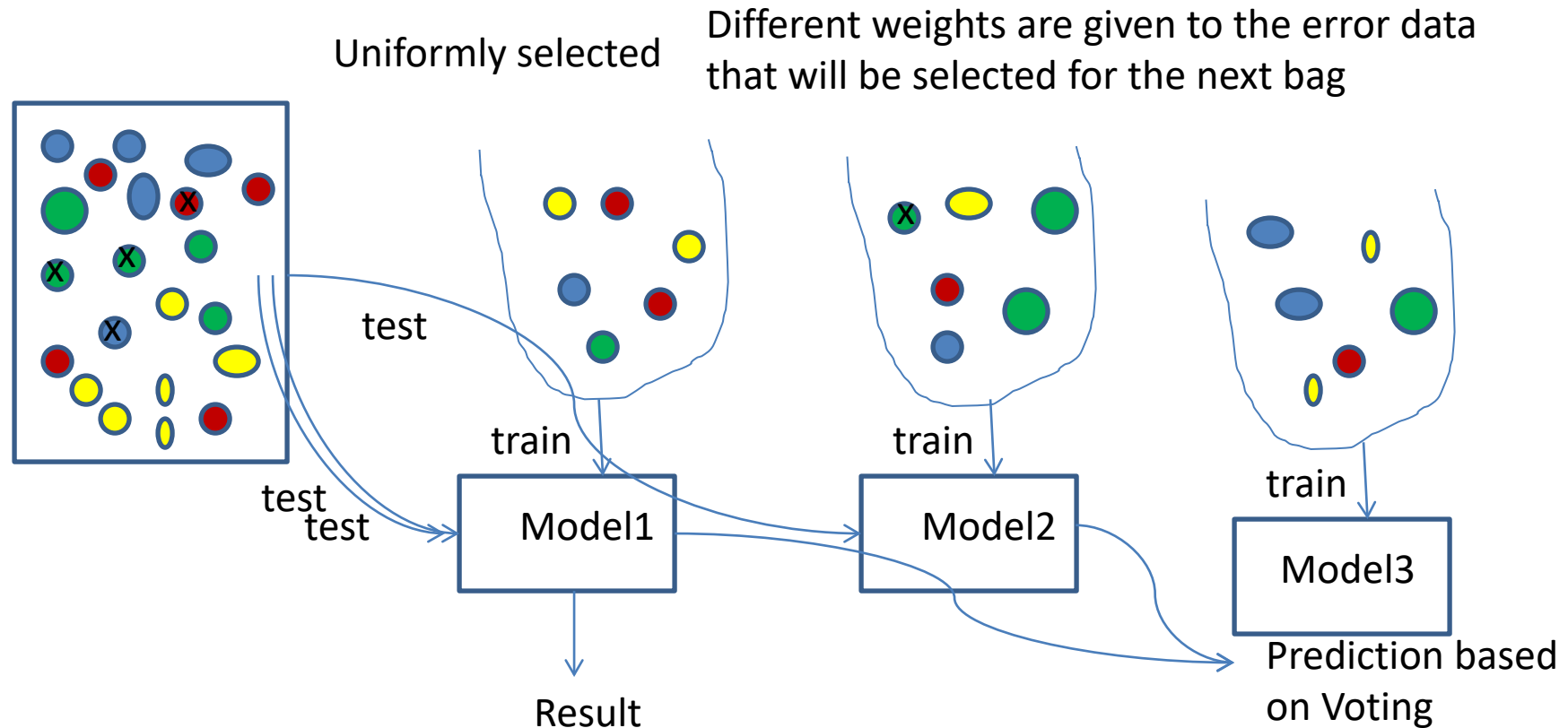
Example



Boosting

- Boosting is an ensemble technique that is similar to bagging, but uses a more sophisticated technique than bootstrap to create diverse training sets.
- Boosting is a sequential process, where each subsequent model attempts to correct the errors of the previous model.
- The succeeding models are dependent on the previous model.

Ada-boosting (Adaptive Boosting)



Error:   

Errors are found using the actual values and predicted values.

Error:   

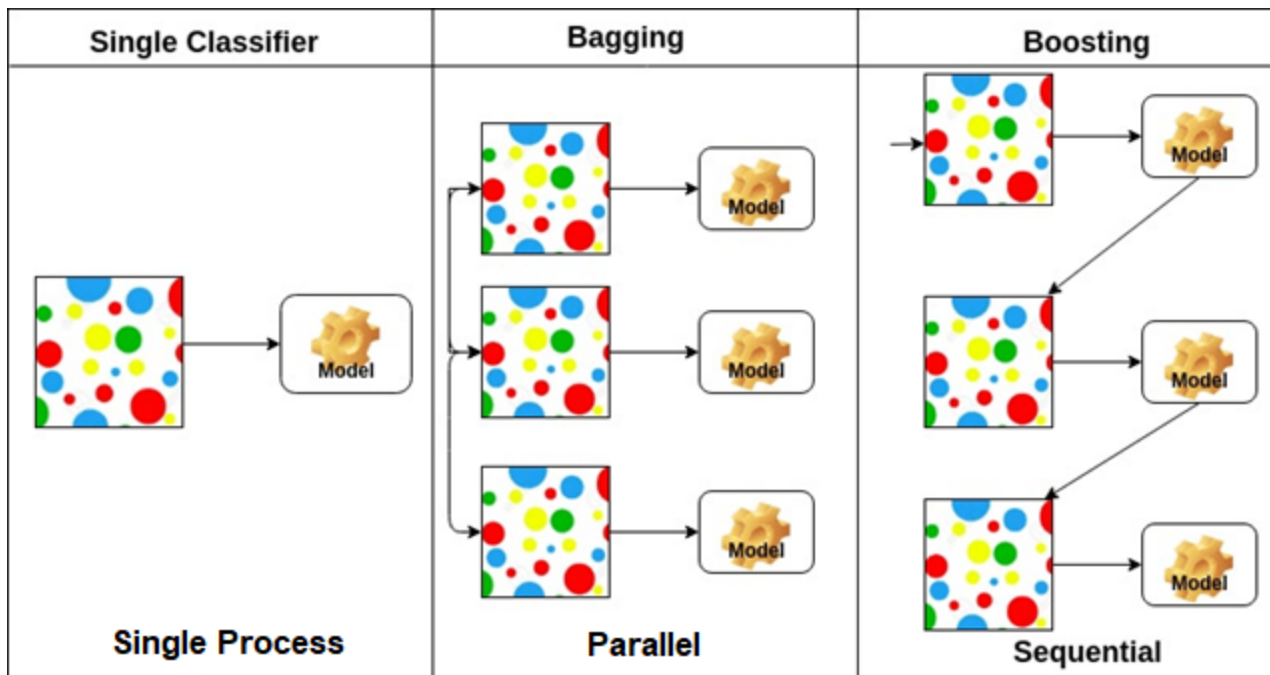
Errors are found using the actual values and predicted values.

A subset is created from the original data set which are incorrectly predicted, and given higher weights. Initially, all data points are given equal weights. A base model is created on this subset. Another model is created and predictions are made on the dataset. This model is used to make predictions on the whole dataset. The final model (strong learner) is the weighted mean of all the models (weak learners).

Important Note:

- As a result, the boosting algorithm combines a number of **weak learners** to form a **strong learner**.
- The individual models would not perform well on the entire dataset, but they work well for some part of the dataset.
- Each model actually boosts the performance of the ensemble.

Cont.



Random Forest

- Random Forest is another ensemble machine learning algorithm that follows the bagging technique (bootstrap aggregating).
- It is an extension of the bagging estimator algorithm. The base estimators in random forest are decision trees.
 - Random subsets are created from the original dataset (bootstrapping).
 - At each node in the decision tree, only a random set of data points are considered to decide the best split.
 - A decision tree model is fitted on each of the subsets.
 - The final prediction is calculated by averaging the predictions from all decision trees.

Random Forest

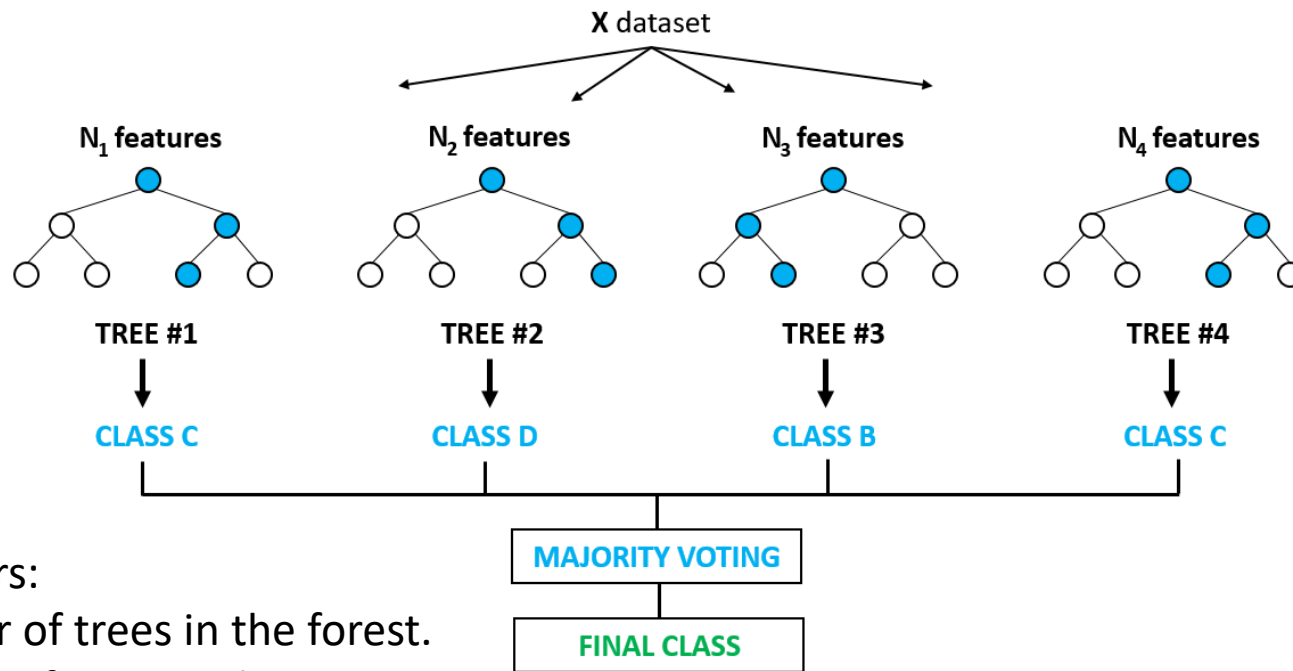
- Supervised based algorithm
 - In a classification problem, each tree votes and the most popular class is chosen as the final result.
 - In a regression problem, the average of all the tree outputs is considered as the final result.
- It can be used both for classification and regression.
- It is simpler and more powerful compared to the other non-linear classification algorithms.
- A forest is comprised of trees.
- Each tree depends on an independent random sample.
- Random forests has a variety of applications, such as recommendation engines, image classification and feature selection.

How Random Forest works?

- It works in four steps:
 - Select random samples from a given dataset.
 - Construct a decision tree for each sample and get a prediction result from each decision tree.
 - Perform a vote for each predicted result.
 - Select the prediction result with the most votes as the final prediction.

Random Forest

- Random forest **randomly** selects data points and features, and builds **multiple trees (Forest)** .



Parameters:

- * Number of trees in the forest.
- * criterion: for example entropy
- * max_depth: The maximum depth of the tree.

Advantages and Disadvantages:

- Advantages:
 - Random forests is considered as a highly accurate and robust method because of the number of decision trees participating in the process.
 - It does not suffer from the overfitting problem.
 - The algorithm can be used in both classification and regression problems.
 - Random forests also offers a good feature selection indicator.
- Disadvantages:
 - Random forests is slow in generating predictions because it has multiple decision trees.
 - The model is difficult to interpret compared to a decision tree.

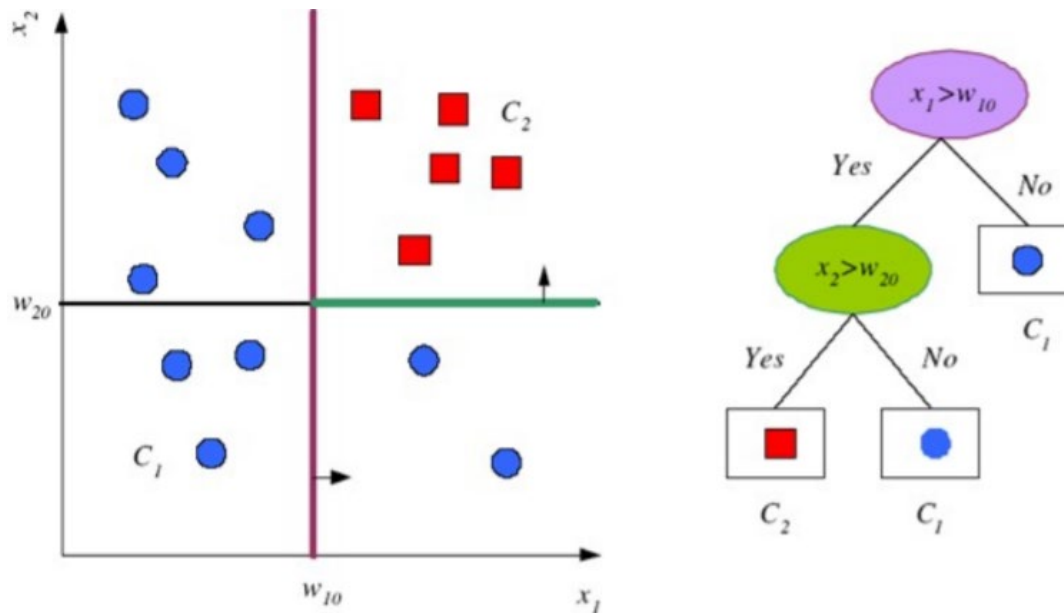
Decision Trees

- One of the most widely used
- Practical
- Approximating discrete-valued target based
- Popular
- Used for classification
- Use nodes, roots (leaves)
- Training based
- Learns decision trees by constructing them **top-down**

Divide and Conquer

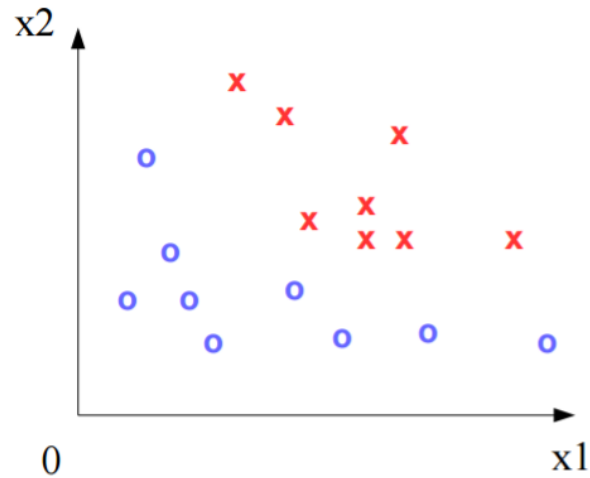
- Internal decision nodes
 - Univariate:
 - Uses a single attribute, x_i
 - Numeric x_i : Binary split : $x_i > w_m$
 - Discrete x_i : n-way split for n possible values
- Leaves
 - Classification: Class labels
 - Regression: Numeric value, average value

Decision Tree Representation

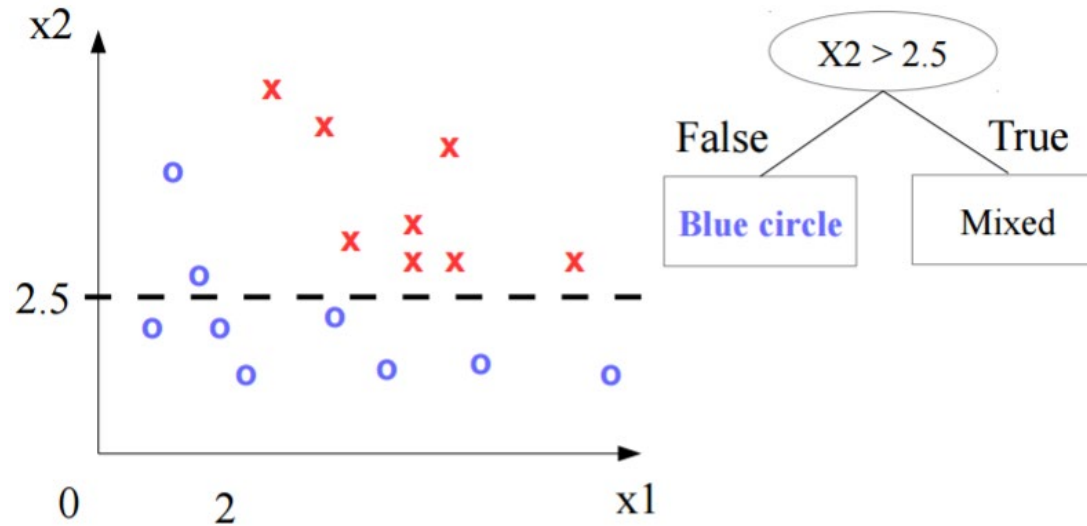


Uses Nodes and Trees

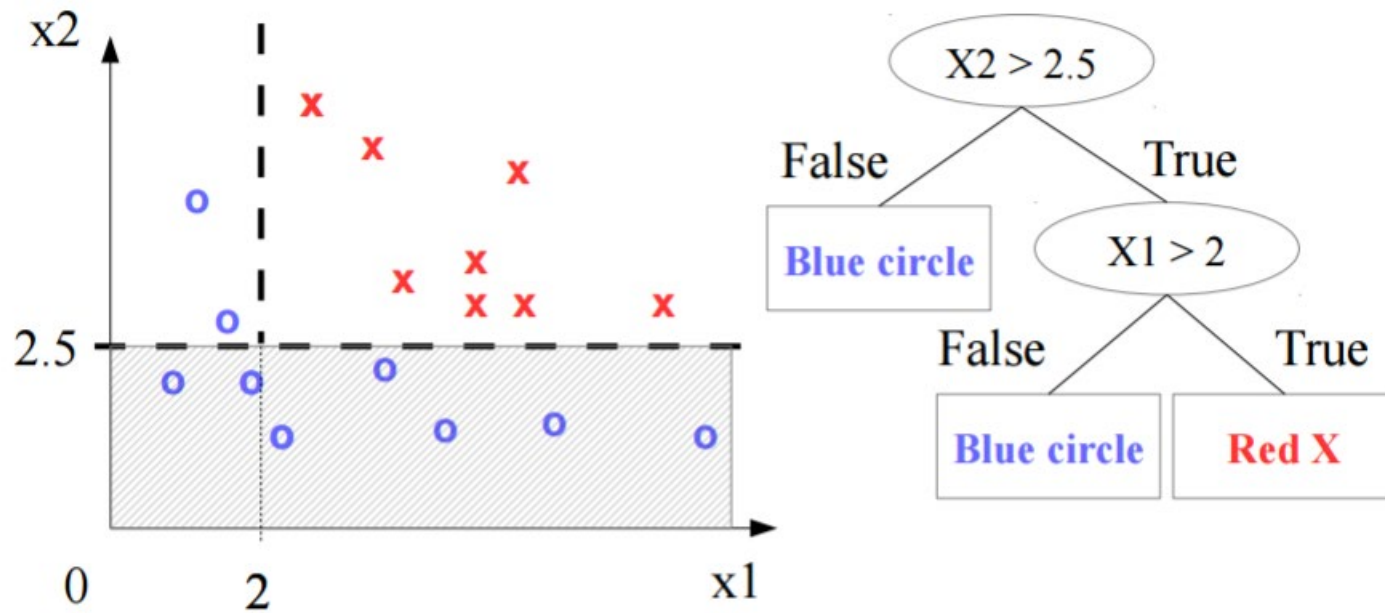
Example 2: Creating a Decision Tree



Example 2: Creating a Decision Tree



Example 2: Creating a Decision Tree

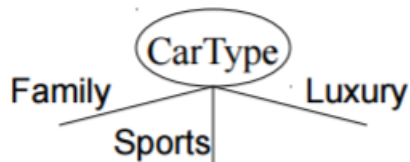


Tree Induction

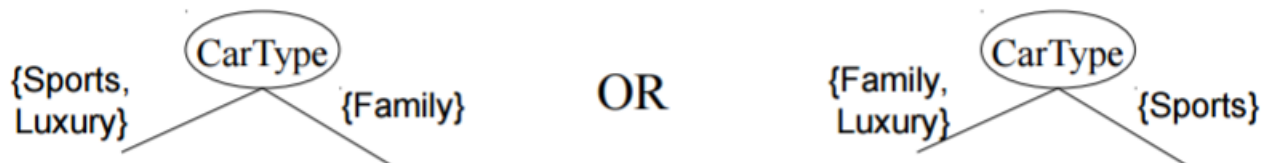
- Greedy strategy
 - Split the records based on an attribute test that optimizes a certain criterion.
- Issues
 - Determine how to split the records
 - Splitting using different attribute types?
 - How to determine the best split?
 - Determine when to stop splitting

Splitting Based on Nominal Attributes

- **Multi-way split:** Use as many partitions as distinct values.



- **Binary split:** Divides values into two subsets.
Need to find optimal partitioning



Entropy

- statistical measure from information theory
- for binary classification: $H(S) \equiv -p_{\oplus} \log_2 p_{\oplus} - p_{\ominus} \log_2 p_{\ominus}$
- for n-ary classification: $H(S) \equiv \sum_{i=1}^n -p_i \log_2 p_i$
- interpretation: specification of the minimum number of bits of information needed to encode the classification of an arbitrary member of S
- alternative: number of yes/no questions

Creating Trees: Best Split

- If node m is pure, generate a leaf and stop, otherwise split and continue recursively
- Impurity after split: N_{mj} of N_m take branch j .
 N_{mj} belong to C_i

$$\hat{p}(C_i | \mathbf{x}, m, j) \equiv p_{mj}^i = \frac{N_{mj}^i}{N_{mj}}$$

$$I'_m = - \sum_{j=1}^n \frac{N_{mj}}{N_m} \sum_{i=1}^K p_{mj}^i \log_2 p_{mj}^i$$

Play Tennis today?

Day	Outlook	Temperatur	Humidity	Wind	PlayTennis?
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

- Outlook:

$$\begin{aligned}
 I'_{root} &= - \sum_{j \in \{Sunny, Overcast, Rain\}} \frac{N_j}{N} \sum_{i \in \{Yes, No\}} p_j^i \log_2 p_j^i \\
 &= - \left[\frac{5}{14} \left(\frac{2}{5} \log_2 \frac{2}{5} + \frac{3}{5} \log_2 \frac{3}{5} \right) + \frac{4}{14} \left(\frac{4}{4} \log_2 \frac{4}{4} + \frac{0}{4} \log_2 \frac{0}{4} \right) + \frac{5}{14} \left(\frac{3}{5} \log_2 \frac{3}{5} + \frac{2}{5} \log_2 \frac{2}{5} \right) \right] \\
 &= 0.693
 \end{aligned}$$

- Temperature:

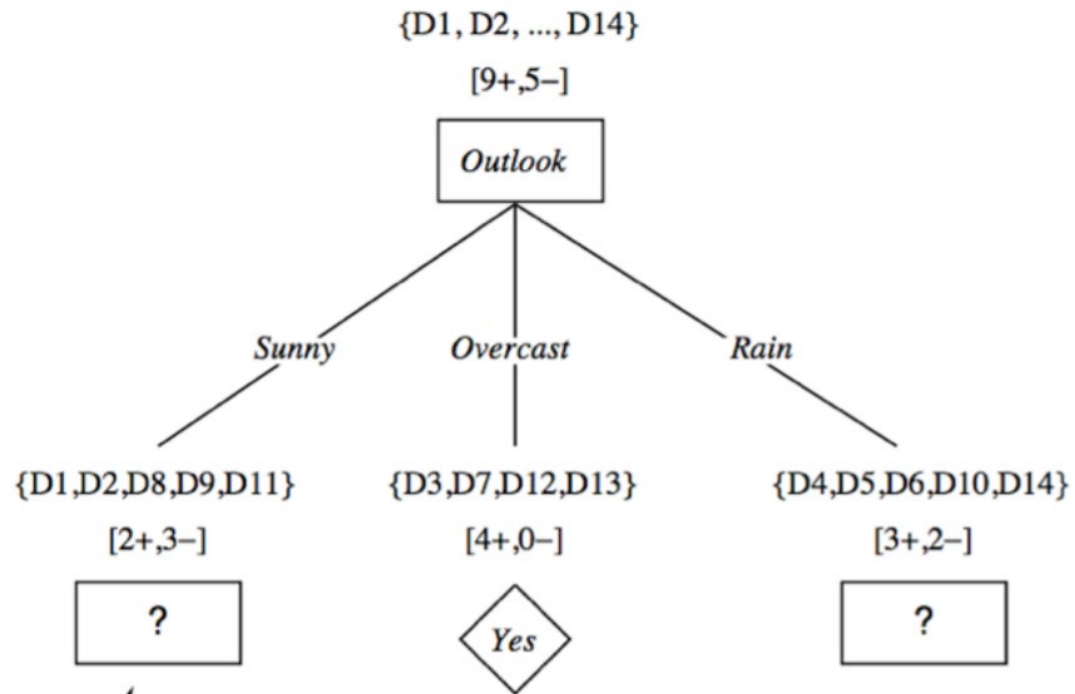
$$\begin{aligned}
 I'_{root} &= - \sum_{j \in \{Hot, Mild, Cool\}} \frac{N_j}{N} \sum_{i \in \{Yes, No\}} p_j^i \log_2 p_j^i \\
 &= - \left[\frac{4}{14} \left(\frac{2}{4} \log_2 \frac{2}{4} + \frac{2}{4} \log_2 \frac{2}{4} \right) + \frac{6}{14} \left(\frac{4}{6} \log_2 \frac{4}{6} + \frac{2}{6} \log_2 \frac{2}{6} \right) + \frac{4}{14} \left(\frac{3}{4} \log_2 \frac{3}{4} + \frac{1}{4} \log_2 \frac{1}{4} \right) \right] \\
 &= 0.915
 \end{aligned}$$

- Humidity:

$$\begin{aligned}
 I'_{root} &= - \sum_{j \in \{High, Normal\}} \frac{N_j}{N} \sum_{i \in \{Yes, No\}} p_j^i \log_2 p_j^i \\
 &= - \left[\frac{7}{14} \left(\frac{3}{7} \log_2 \frac{3}{7} + \frac{4}{7} \log_2 \frac{4}{7} \right) + \frac{7}{14} \left(\frac{6}{7} \log_2 \frac{6}{7} + \frac{1}{7} \log_2 \frac{1}{7} \right) \right] \\
 &= 0.789
 \end{aligned}$$

- Wind

$$\begin{aligned}
 I'_{root} &= - \sum_{j \in \{Strong, Weak\}} \frac{N_j}{N} \sum_{i \in \{Yes, No\}} p_j^i \log_2 p_j^i \\
 &= - \left[\frac{6}{14} \left(\frac{3}{6} \log_2 \frac{3}{6} + \frac{3}{6} \log_2 \frac{3}{6} \right) + \frac{8}{14} \left(\frac{6}{8} \log_2 \frac{6}{8} + \frac{2}{8} \log_2 \frac{2}{8} \right) \right] \\
 &= 0.892
 \end{aligned}$$



Which attribute should be tested here?

$$X_{\text{Sunny}} = \{D_1, D_2, D_8, D_9, D_{11}\}$$

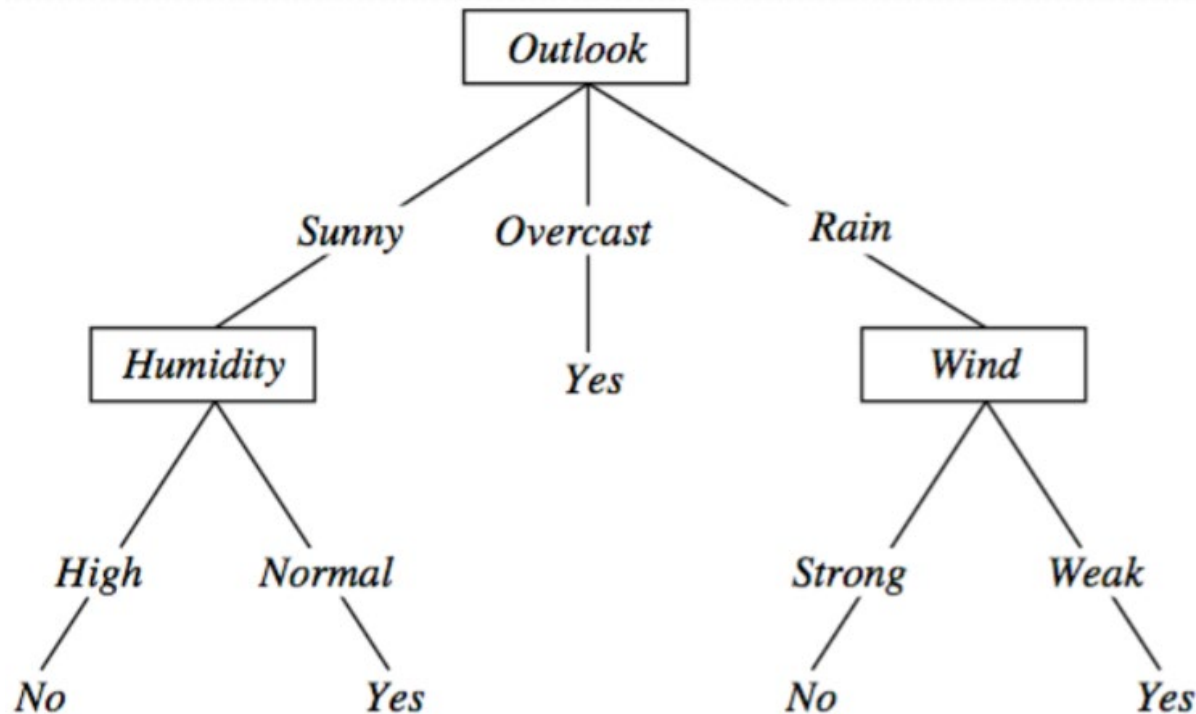
Day	Outlook	Temperatur	Humidity	Wind	PlayTennis?
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes

Humidity:

$$\begin{aligned}
 I'_{\text{sunny,humidity}} &= - \sum_{j \in \{High, Normal\}} \frac{N_j}{N} \sum_{i \in \{Yes, No\}} p_j^i \log_2 p_j^i \\
 &= - \left[\frac{3}{5} \left(\frac{0}{3} \log_2 \frac{0}{3} + \frac{3}{3} \log_2 \frac{3}{3} \right) + \frac{2}{5} \left(\frac{2}{2} \log_2 \frac{2}{2} + \frac{0}{2} \log_2 \frac{0}{2} \right) \right] \\
 &= 0
 \end{aligned}$$

- Do the same calculation for Temperature and Wind
- Humidity has lowest entropy, so Humidity is split attribute at Sunny node

Final Induced Tree



This decision tree is equivalent to:

if ($Outlook = Sunny$) $\wedge$ ($Humidity = Normal$) **then** *Yes*;

if ($Outlook = Overcast$) **then** *Yes*;

if ($Outlook = Rain$) $\wedge$ ($Wind = Weak$) **then** *Yes*;

Observations

- Temperature was not needed
 - Decision tree can also be used for feature extraction (a.k.a.) feature selection)
 - Tree is short
 - Occam's Razor: Shorter trees are better

Random Forests vs Decision Trees

- Random forests is a set of multiple decision trees.
- Decision trees may suffer from overfitting, but random forests prevents overfitting by creating trees on random subsets.
- Decision trees are computationally faster.